

brain_tumor_mri_resnet_pipeline

July 9, 2026

1 Brain Tumor MRI Classification with Deep Learning

One of the most pervasive problems in health are cancers and tumours. They can appear in many parts of the body and on many organs, but it is imperative that their detection be swift in order for treatments to start early and be effective, or for the removal of the tumor before it becomes unmanageable and threatening to a person's life. This notebook implements an end-to-end analytical pipeline for classifying brain MRI scans as tumor (**yes**) or no tumor (**no**). It uses the dataset folder `archive/brain_tumor_dataset`, removes exact duplicate images before splitting, compares three deep learning model variations, and produces report-ready evaluation outputs.

1.1 Objectives

The primary objective is to build a supervised image classification model that can identify whether a brain MRI scan shows evidence of a tumor. The analysis is designed as a proof-of-concept decision-support workflow rather than a clinical diagnostic system.

Specific objectives:

- Create a duplicate-safe modeling dataset from the retained `archive/brain_tumor_dataset` folder.
- Train and compare three deep learning models that can aid us in accomplishing our goal.
- Evaluate models using accuracy, precision, recall, F1-score, ROC-AUC, and confusion matrices.
- Recommend a final model based on validation performance, with emphasis on recall and F1-score.
- Produce report-ready findings, limitations, and next steps.

Thus, this analysis pretends to not only develop a reasonably good model for tumor detection in MRI scans, but to also provide a full workflow where we explore the data and perform all necessary transformations on it to aid our models perform better. At the end, we will have also obtained a working model for our purpose and based on it we will propose further clarifications, steps, and details about our endeavor.

1.2 Data Description

The dataset contains brain MRI image files organized into two binary class folders:

- **yes**: MRI scans labeled as showing a brain tumor.
- **no**: MRI scans labeled as not showing a brain tumor.

Only `archive/brain_tumor_dataset` is used for modeling. The raw retained dataset contains 253 images: 155 tumor images and 98 no-tumor images. Exact duplicate removal is performed before train/validation/test splitting. As we will see in a cell below, this will leave us with 228 unique images: 141 tumor images and 87 no-tumor images.

The dataset includes image files only. It does not include patient demographics, scanner metadata, tumor type, tumor grade, tumor location annotations, segmentation masks, or clinical outcomes. As a result, this analysis is limited to binary image classification.

1.3 Environment Setup

Run the install cell only if TensorFlow is missing in your current environment.

```
[ ]: # Uncomment and run if TensorFlow is not installed.  
# %pip install tensorflow
```

```
[8]: from pathlib import Path  
import hashlib  
import json  
import random  
import warnings  
  
import numpy as np  
import pandas as pd  
import matplotlib.pyplot as plt  
import seaborn as sns  
from PIL import Image, ImageOps, UnidentifiedImageError  
  
from sklearn.model_selection import train_test_split  
from sklearn.utils.class_weight import compute_class_weight  
from sklearn.metrics import (  
    accuracy_score,  
    precision_score,  
    recall_score,  
    f1_score,  
    roc_auc_score,  
    confusion_matrix,  
    classification_report,  
)  
  
try:  
    import tensorflow as tf  
    from tensorflow import keras  
    from tensorflow.keras import layers  
    from tensorflow.keras.applications import ResNet50  
    from tensorflow.keras.applications.resnet50 import preprocess_input  
except ModuleNotFoundError as exc:  
    raise ModuleNotFoundError(
```

```

        "TensorFlow/Keras is required for model training. Run the previous_
↪install cell, "
        "restart the kernel, and rerun the notebook."
    ) from exc

warnings.filterwarnings("ignore")
sns.set_theme(style="whitegrid")

SEED = 42
random.seed(SEED)
np.random.seed(SEED)
tf.random.set_seed(SEED)

PROJECT_ROOT = Path.cwd()
DATA_DIR = PROJECT_ROOT / "archive" / "brain_tumor_dataset"
OUTPUT_DIR = PROJECT_ROOT / "outputs"
OUTPUT_DIR.mkdir(exist_ok=True)

IMG_SIZE = (224, 224)
BATCH_SIZE = 16
AUTOTUNE = tf.data.AUTOTUNE
LABEL_MAP = {"no": 0, "yes": 1}
CLASS_NAMES = ["no", "yes"]

print(f"TensorFlow version: {tf.__version__}")
print(f"Dataset path: {DATA_DIR}")

```

TensorFlow version: 2.21.0

Dataset path: C:\Users\adrian\Documents\IBM_Projects\DeepLearning_Project\archive\brain_tumor_dataset

1.4 Dataset Inventory and Duplicate-Safe Cleaning

Since there is a possibility that some images are duplicated, we want to remove them so that the model can train on different data and not overfit on specific patterns in the training data. Exact duplicates are identified by SHA-256 hash and removed from the modeling DataFrame before splitting. The original image files are not deleted.

```

[9]: def sha256_file(path: Path, block_size: int = 1 << 20) -> str:
    digest = hashlib.sha256()
    with path.open("rb") as file:
        for block in iter(lambda: file.read(block_size), b""):
            digest.update(block)
    return digest.hexdigest()

def inspect_image(path: Path) -> dict:
    record = {

```

```

        "path": str(path),
        "relative_path": str(path.relative_to(PROJECT_ROOT)),
        "file_name": path.name,
        "extension": path.suffix.lower(),
        "file_size_bytes": path.stat().st_size,
        "label_name": path.parent.name,
        "label": LABEL_MAP.get(path.parent.name),
        "readable": False,
        "width": np.nan,
        "height": np.nan,
        "aspect_ratio": np.nan,
        "sha256": None,
        "error": None,
    }
    try:
        with Image.open(path) as image:
            image = ImageOps.exif_transpose(image).convert("RGB")
            record["width"], record["height"] = image.size
            record["aspect_ratio"] = record["width"] / record["height"]
            record["sha256"] = sha256_file(path)
            record["readable"] = True
    except (UnidentifiedImageError, OSError, ValueError) as exc:
        record["error"] = repr(exc)
    return record

image_paths = sorted(
    [p for p in DATA_DIR.rglob("*") if p.suffix.lower() in {".jpg", ".jpeg", ".
    ↪png"}]
)

metadata = pd.DataFrame([inspect_image(path) for path in image_paths])
metadata.to_csv(OUTPUT_DIR / "raw_image_metadata.csv", index=False)

print(f"Raw image files found: {len(metadata)}")
display(metadata.head())
display(metadata.groupby(["label_name", "readable"]).size().rename("count").
    ↪reset_index())

```

Raw image files found: 253

```

                                path \
0  C:\Users\adrian\Documents\IBM_Projects\DeepLea...
1  C:\Users\adrian\Documents\IBM_Projects\DeepLea...
2  C:\Users\adrian\Documents\IBM_Projects\DeepLea...
3  C:\Users\adrian\Documents\IBM_Projects\DeepLea...
4  C:\Users\adrian\Documents\IBM_Projects\DeepLea...

```

	relative_path	file_name	extension	\
0	archive\brain_tumor_dataset\no\1	no.jpeg	1 no.jpeg	.jpeg
1	archive\brain_tumor_dataset\no\10	no.jpg	10 no.jpg	.jpg
2	archive\brain_tumor_dataset\no\11	no.jpg	11 no.jpg	.jpg
3	archive\brain_tumor_dataset\no\12	no.jpg	12 no.jpg	.jpg
4	archive\brain_tumor_dataset\no\13	no.jpg	13 no.jpg	.jpg

	file_size_bytes	label_name	label	readable	width	height	aspect_ratio	\
0	54521	no	0	True	630	630	1.000000	
1	3848	no	0	True	173	201	0.860697	
2	3475	no	0	True	300	168	1.785714	
3	4142	no	0	True	275	183	1.502732	
4	4570	no	0	True	300	168	1.785714	

	sha256	error
0	e4d9d83f89339366f4f77137d440a4c3b62e799a482ebf...	None
1	3c3bdbcf86cd640b0efb1f4a95726f65664a2b760cf385...	None
2	ab01efa9ce7ad51de522ba8cfa0de7d46ce3332215227...	None
3	91eb18f61d813028c6de3d9a78b79f807712662d006287...	None
4	c8280173142dbe052a428603990c3124edc9700d71fc33...	None

	label_name	readable	count
0	no	True	98
1	yes	True	155

```
[10]: invalid_labels = sorted(set(metadata["label_name"]) - set(LABEL_MAP))
if invalid_labels:
    raise ValueError(f"Unexpected label folders found: {invalid_labels}")

if not metadata["readable"].all():
    display(metadata.loc[~metadata["readable"], ["relative_path", "error"]])
    raise ValueError("Unreadable images found. Review the files above before_
↪training.")

cross_label_duplicates = (
    metadata.groupby("sha256")["label_name"]
    .nunique()
    .loc[lambda series: series > 1]
)

if len(cross_label_duplicates) > 0:
    duplicate_hashes = cross_label_duplicates.index.tolist()
    display(metadata.loc[metadata["sha256"].isin(duplicate_hashes),
↪["relative_path", "label_name", "sha256"]])
    raise ValueError("Exact duplicate images appear across both labels. Resolve_
↪labels before modeling.")

clean_metadata = (
```

```

metadata.sort_values(["label_name", "relative_path"])
.drop_duplicates(subset="sha256", keep="first")
.reset_index(drop=True)
)

summary = pd.DataFrame(
    {
        "stage": ["raw", "deduplicated"],
        "total_images": [len(metadata), len(clean_metadata)],
        "yes_images": [int((metadata["label_name"] == "yes").sum()),
↪int((clean_metadata["label_name"] == "yes").sum())],
        "no_images": [int((metadata["label_name"] == "no").sum()),
↪int((clean_metadata["label_name"] == "no").sum())],
    }
)

summary["duplicates_removed"] = summary.loc[0, "total_images"] -
↪summary["total_images"]

clean_metadata.to_csv(OUTPUT_DIR / "clean_image_metadata.csv", index=False)
summary.to_csv(OUTPUT_DIR / "dataset_cleaning_summary.csv", index=False)

display(summary)
print(f"Exact duplicate files removed from modeling DataFrame: {len(metadata) -
↪len(clean_metadata)}")

```

	stage	total_images	yes_images	no_images	duplicates_removed
0	raw	253	155	98	0
1	deduplicated	228	141	87	25

Exact duplicate files removed from modeling DataFrame: 25

1.5 Exploratory Data Analysis

This section documents class balance, image-size variability, and examples from each class. These checks justify resizing, stratified splitting, class weighting, and duplicate-safe evaluation, especially in the context of our small dataset.

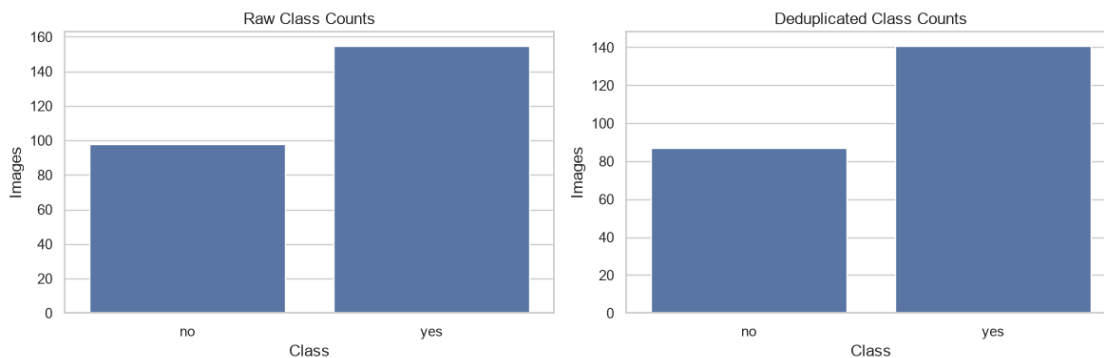
We also perform a brief graphical visualization of our images when it comes to their dimensions. Because the sources of the images can be different, whether that be from different institutions or instrumentations employed for performing the MRI scans, we may need to perform some data augmentation in order for our images to have more or less consistent sizes, or we may need to take into account these differences in dimensions in our models.

Lastly, we show some samples of images, both labeled as not presenting a tumor and presenting one. We can see on first glance that the tumor is usually relatively easy to identify, as specific clumps and masses are detected within the brain itself. The objective is thus to train a model that can accurately find if an image presents these clumps.

```
[11]: fig, axes = plt.subplots(1, 2, figsize=(12, 4))
sns.countplot(data=metadata, x="label_name", order=CLASS_NAMES, ax=axes[0])
axes[0].set_title("Raw Class Counts")
axes[0].set_xlabel("Class")
axes[0].set_ylabel("Images")

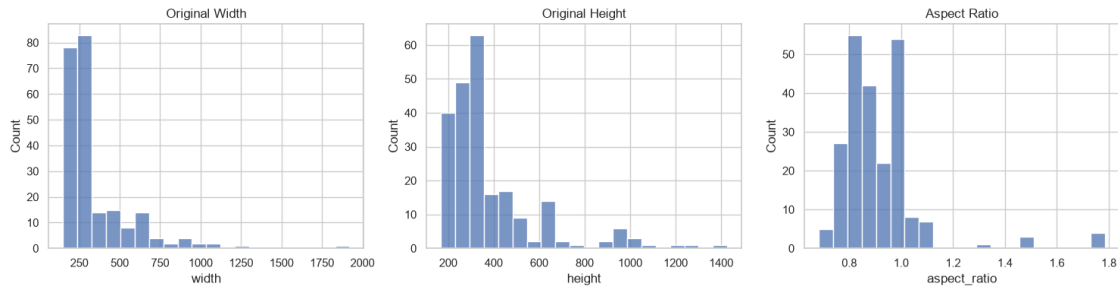
sns.countplot(data=clean_metadata, x="label_name", order=CLASS_NAMES,
               ↪ax=axes[1])
axes[1].set_title("Deduplicated Class Counts")
axes[1].set_xlabel("Class")
axes[1].set_ylabel("Images")

plt.tight_layout()
plt.savefig(OUTPUT_DIR / "class_counts.png", dpi=160, bbox_inches="tight")
plt.show()
```



```
[12]: fig, axes = plt.subplots(1, 3, figsize=(15, 4))
sns.histplot(clean_metadata["width"], bins=20, ax=axes[0])
axes[0].set_title("Original Width")
sns.histplot(clean_metadata["height"], bins=20, ax=axes[1])
axes[1].set_title("Original Height")
sns.histplot(clean_metadata["aspect_ratio"], bins=20, ax=axes[2])
axes[2].set_title("Aspect Ratio")
plt.tight_layout()
plt.savefig(OUTPUT_DIR / "image_dimension_distributions.png", dpi=160,
           ↪bbox_inches="tight")
plt.show()

display(clean_metadata[["width", "height", "aspect_ratio", "file_size_bytes"]].
        ↪describe())
```



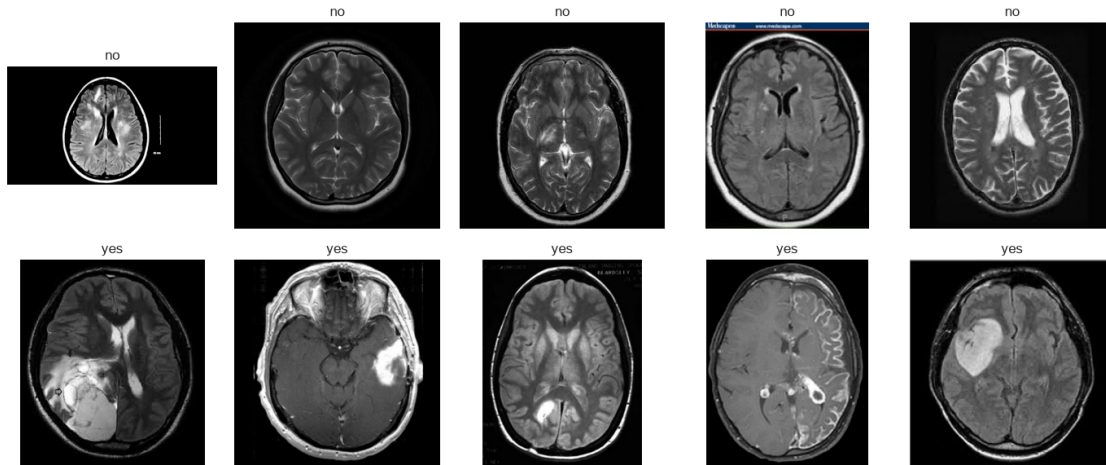
	width	height	aspect_ratio	file_size_bytes
count	228.000000	228.000000	228.000000	228.000000
mean	352.978070	384.956140	0.916865	34755.925439
std	219.290736	212.954128	0.165929	63043.406114
min	150.000000	168.000000	0.682454	3475.000000
25%	225.000000	246.750000	0.816138	7801.750000
50%	276.000000	332.500000	0.877640	20857.000000
75%	394.750000	442.000000	1.000000	31267.500000
max	1920.000000	1427.000000	1.785714	727906.000000

```
[13]: def show_samples(df: pd.DataFrame, n_per_class: int = 5) -> None:
    class_samples = {}
    for label_name in CLASS_NAMES:
        label_df = df.loc[df["label_name"] == label_name]
        if label_df.empty:
            raise ValueError(f"No images found for class: {label_name}")
        class_samples[label_name] = label_df.sample(
            n=min(n_per_class, len(label_df)),
            random_state=SEED,
        )

    fig, axes = plt.subplots(len(CLASS_NAMES), n_per_class, figsize=(14, 3 *
    len(CLASS_NAMES)))
    axes = np.atleast_2d(axes)
    for ax in axes.ravel():
        ax.axis("off")
    for row_idx, label_name in enumerate(CLASS_NAMES):
        label_samples = class_samples[label_name]
        for col_idx, (_, row) in enumerate(label_samples.iterrows()):
            with Image.open(row["path"]) as image:
                axes[row_idx, col_idx].imshow(ImageOps.exif_transpose(image).
                convert("RGB"), cmap="gray")
            axes[row_idx, col_idx].set_title(label_name)
    plt.tight_layout()
    plt.savefig(OUTPUT_DIR / "class_sample_images.png", dpi=160,
    bbox_inches="tight")
```

```
plt.show()
```

```
show_samples(clean_metadata)
```



1.6 Split Strategy and TensorFlow Input Pipeline

Splitting happens after deduplication, with stratification by class due to the reduced size of the dataset. The notebook verifies that the same image hash does not appear in more than one split.

```
[14]: train_df, temp_df = train_test_split(
    clean_metadata,
    test_size=0.30,
    stratify=clean_metadata["label"],
    random_state=SEED,
)

val_df, test_df = train_test_split(
    temp_df,
    test_size=0.50,
    stratify=temp_df["label"],
    random_state=SEED,
)

for split_name, split_df in [("train", train_df), ("validation", val_df),
    ("test", test_df)]:
    split_df = split_df.copy().reset_index(drop=True)
    split_df.to_csv(OUTPUT_DIR / f"{split_name}_split.csv", index=False)
    print(f"{split_name}: {len(split_df)} images")
    display(split_df["label_name"].value_counts().rename_axis("class").
    reset_index(name="count"))
```

```

split_hashes = {
    "train": set(train_df["sha256"]),
    "validation": set(val_df["sha256"]),
    "test": set(test_df["sha256"]),
}
assert split_hashes["train"].isdisjoint(split_hashes["validation"])
assert split_hashes["train"].isdisjoint(split_hashes["test"])
assert split_hashes["validation"].isdisjoint(split_hashes["test"])

class_weights_array = compute_class_weight(
    class_weight="balanced",
    classes=np.array([0, 1]),
    y=train_df["label"].to_numpy(),
)
CLASS_WEIGHTS = {class_idx: float(weight) for class_idx, weight in zip([0, 1],
↪class_weights_array)}
print(f"Class weights: {CLASS_WEIGHTS}")

```

train: 159 images

	class	count
0	yes	98
1	no	61

validation: 34 images

	class	count
0	yes	21
1	no	13

test: 35 images

	class	count
0	yes	22
1	no	13

Class weights: {0: 1.3032786885245902, 1: 0.8112244897959183}

1.7 Data Augmentation

The main limitation in this analysis is that our dataset is currently very small, with 228 unique images. Ideally, we want more. However, collecting medical data is a severely restricted process, with a lot of ethical concerns and data privacy regulations around it. Data for training is very difficult to come by. Moreover, since in our particular problem we are treating a condition that can realistically threaten a person's life, generating synthetic data can pollute our dataset by creating patterns that may not naturally present themselves in MRI scans where a true tumor exists. For these reasons, we can employ data augmentation.

Data augmentation in image recognition consists of taking our current images and performing very small transformations on them and treating them like a separate image. This can mean rotating

the image, zooming it, translating or increasing or decreasing the contrast (especially useful for grayscale images like MRI scans). These transformations need to be small and retain the critical information and layout of the image. Nevertheless, by adding these small variations we increase our dataset and the variety of the data in it, which should help the models learn specific features instead of learning naive pixel comparisons or very basic features that are not informative.

With this process in mind, we create our splits: train, test and validation, retaining the class proportions and performing data augmentation on them. Data augmentation is only performed on the training set, since that is when the model needs to learn the features it requires for its goal. We also create a variant for CNN and one for ResNet simply because CNN expects its pixels to be normalized between 0 and 1, something that ResNet does not demand.

```
[15]: augmentation = keras.Sequential([
    layers.RandomRotation(0.05, seed=SEED),
    layers.RandomZoom(0.10, seed=SEED),
    layers.RandomTranslation(0.05, 0.05, seed=SEED),
    layers.RandomContrast(0.10, seed=SEED),
],
name="training_augmentation",
)

def load_image(path, label, model_family: str, training: bool = False):
    image = tf.io.read_file(path)
    image = tf.io.decode_image(image, channels=3, expand_animations=False)
    image = tf.image.resize(image, IMG_SIZE)
    image = tf.cast(image, tf.float32)

    if training:
        image = augmentation(image)

    if model_family == "cnn":
        image = image / 255.0
    elif model_family == "resnet":
        image = preprocess_input(image)
    else:
        raise ValueError(f"Unknown model family: {model_family}")

    return image, tf.cast(label, tf.float32)

def make_dataset(df: pd.DataFrame, model_family: str, training: bool = False):
    paths = df["path"].astype(str).to_numpy()
    labels = df["label"].astype(np.float32).to_numpy()
    dataset = tf.data.Dataset.from_tensor_slices((paths, labels))
    if training:
```

```

        dataset = dataset.shuffle(buffer_size=len(df), seed=SEED,
↪reshuffle_each_iteration=True)
        dataset = dataset.map(
            lambda path, label: load_image(path, label, model_family=model_family,
↪training=training),
            num_parallel_calls=AUTOTUNE,
        )
        return dataset.batch(BATCH_SIZE).prefetch(AUTOTUNE)

train_cnn_ds = make_dataset(train_df, "cnn", training=True)
val_cnn_ds = make_dataset(val_df, "cnn", training=False)
test_cnn_ds = make_dataset(test_df, "cnn", training=False)

train_resnet_ds = make_dataset(train_df, "resnet", training=True)
val_resnet_ds = make_dataset(val_df, "resnet", training=False)
test_resnet_ds = make_dataset(test_df, "resnet", training=False)

```

1.8 Model Selection, Evaluation and Definition

We will train and test the following models:

- CNN: acts as our baseline performance. Very appropriate for image recognition and relatively good interpretability compared to other deep learning models.
- Frozen ResNet50: we use a Transfer Learning approach by freezing a ResNet50 pretrained on ImageNet. We simply add as a head a new binary classifier trained exclusively on the MRI scan images.
- Fine tuned ResNet50: we use the pretrained model but we freeze everything except the last residual layer of the ResNet50 model and allow it to train on our MRI scans with a slow learning rate so that it can adapt to the specific features of our dataset.

Other architectures were considered when it comes to CNNs, but we prefer ResNet50 because due to our limited dataset we want to have a deep training layer pipeline without incurring the risk of vanishing gradients. This is something ResNet naturally excels at. Other forms of deep learning such as Autoencoders and their variations have been considered but deemed more inappropriate and less interpretable than a CNN architecture for this problem.

Since this is a classification problem, we prioritize those models with F1-Score and Recall the most, as accuracy alone can be misleading in our case.

```

[16]: def compile_binary_model(model: keras.Model, learning_rate: float) -> keras.
↪Model:
        model.compile(
            optimizer=keras.optimizers.Adam(learning_rate=learning_rate),
            loss="binary_crossentropy",
            metrics=[
                keras.metrics.BinaryAccuracy(name="accuracy"),
                keras.metrics.Precision(name="precision"),
                keras.metrics.Recall(name="recall"),
            ]
        )

```

```

        keras.metrics.AUC(name="auc"),
    ],
)
return model

def build_baseline_cnn() -> keras.Model:
    inputs = keras.Input(shape=(*IMG_SIZE, 3))
    x = layers.Conv2D(32, 3, activation="relu", padding="same")(inputs)
    x = layers.MaxPooling2D()(x)
    x = layers.Conv2D(64, 3, activation="relu", padding="same")(x)
    x = layers.MaxPooling2D()(x)
    x = layers.Conv2D(128, 3, activation="relu", padding="same")(x)
    x = layers.MaxPooling2D()(x)
    x = layers.GlobalAveragePooling2D()(x)
    x = layers.Dropout(0.40)(x)
    x = layers.Dense(64, activation="relu")(x)
    x = layers.Dropout(0.30)(x)
    outputs = layers.Dense(1, activation="sigmoid")(x)
    model = keras.Model(inputs, outputs, name="baseline_cnn")
    return compile_binary_model(model, learning_rate=1e-3)

def build_resnet_feature_extractor() -> keras.Model:
    base_model = ResNet50(weights="imagenet", include_top=False,
    ↪input_shape=(*IMG_SIZE, 3))
    base_model.trainable = False
    inputs = keras.Input(shape=(*IMG_SIZE, 3))
    x = base_model(inputs, training=False)
    x = layers.GlobalAveragePooling2D()(x)
    x = layers.Dropout(0.40)(x)
    x = layers.Dense(128, activation="relu")(x)
    x = layers.Dropout(0.30)(x)
    outputs = layers.Dense(1, activation="sigmoid")(x)
    model = keras.Model(inputs, outputs, name="resnet50_frozen")
    return compile_binary_model(model, learning_rate=1e-3)

def prepare_resnet_for_fine_tuning(model: keras.Model) -> keras.Model:
    base_model = next(layer for layer in model.layers if isinstance(layer,
    ↪keras.Model) and layer.name.startswith("resnet50"))
    base_model.trainable = True

    for layer in base_model.layers:
        layer.trainable = layer.name.startswith("conv5_")
        if isinstance(layer, layers.BatchNormalization):
            layer.trainable = False

```

```

    return compile_binary_model(model, learning_rate=1e-5)

baseline_cnn = build_baseline_cnn()
resnet_frozen = build_resnet_feature_extractor()

baseline_cnn.summary()
resnet_frozen.summary()

```

WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.

Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/resnet/resnet50_weights_tf_dim_ordering_tf_kernels_notop.h5
94765736/94765736 3s

0us/step

Model: "baseline_cnn"

Layer (type) ↳Param #	Output Shape	
input_layer_1 (InputLayer) ↳ 0	(None, 224, 224, 3)	↳
conv2d (Conv2D) ↳896	(None, 224, 224, 32)	↳
max_pooling2d (MaxPooling2D) ↳ 0	(None, 112, 112, 32)	↳
conv2d_1 (Conv2D) ↳18,496	(None, 112, 112, 64)	↳
max_pooling2d_1 (MaxPooling2D) ↳ 0	(None, 56, 56, 64)	↳
conv2d_2 (Conv2D) ↳73,856	(None, 56, 56, 128)	↳
max_pooling2d_2 (MaxPooling2D) ↳ 0	(None, 28, 28, 128)	↳
global_average_pooling2d ↳ 0	(None, 128)	↳

(GlobalAveragePooling2D)		
↪		
dropout (Dropout)	(None, 128)	
↪ 0		
dense (Dense)	(None, 64)	
↪ 8,256		
dropout_1 (Dropout)	(None, 64)	
↪ 0		
dense_1 (Dense)	(None, 1)	
↪ 65		

Total params: 101,569 (396.75 KB)

Trainable params: 101,569 (396.75 KB)

Non-trainable params: 0 (0.00 B)

Model: "resnet50_frozen"

Layer (type)	Output Shape	
↪ Param #		
input_layer_3 (InputLayer)	(None, 224, 224, 3)	
↪ 0		
resnet50 (Functional)	(None, 7, 7, 2048)	
↪ 23,587,712		
global_average_pooling2d_1	(None, 2048)	
↪ 0		
(GlobalAveragePooling2D)		
↪		
dropout_2 (Dropout)	(None, 2048)	
↪ 0		
dense_2 (Dense)	(None, 128)	
↪ 262,272		

dropout_3 (Dropout) (None, 128) └
↪ 0

dense_3 (Dense) (None, 1) └
↪ 129

Total params: 23,850,113 (90.98 MB)

Trainable params: 262,401 (1.00 MB)

Non-trainable params: 23,587,712 (89.98 MB)

1.9 Model Training

Early stopping and checkpointing reduce overfitting risk. Even if we train the model for more epochs before the early stopping method halts the model's training, we will restore the model to the best performing epoch based on an increase or stagnation of the validation loss. The final ResNet model is fine-tuned only after the frozen transfer-learning model has converged to make sure it is learning properly

```
[17]: def training_callbacks(model_name: str):  
    checkpoint_path = OUTPUT_DIR / f"{model_name}.keras"  
    return [  
        keras.callbacks.EarlyStopping(  
            monitor="val_loss",  
            patience=5,  
            restore_best_weights=True,  
            verbose=1,  
        ),  
        keras.callbacks.ReduceLROnPlateau(  
            monitor="val_loss",  
            factor=0.3,  
            patience=3,  
            min_lr=1e-7,  
            verbose=1,  
        ),  
        keras.callbacks.ModelCheckpoint(  
            checkpoint_path,  
            monitor="val_loss",  
            save_best_only=True,  
            verbose=1,  
        ),  
    ]
```

```

def train_model(model, train_ds, val_ds, model_name: str, epochs: int):
    print(f"Training {model_name}")
    history = model.fit(
        train_ds,
        validation_data=val_ds,
        epochs=epochs,
        class_weight=CLASS_WEIGHTS,
        callbacks=training_callbacks(model_name),
        verbose=1,
    )
    history_df = pd.DataFrame(history.history)
    history_df.to_csv(OUTPUT_DIR / f"{model_name}_history.csv", index=False)
    return history

history_baseline = train_model(baseline_cnn, train_cnn_ds, val_cnn_ds,
    ↪ "baseline_cnn", epochs=25)
history_resnet_frozen = train_model(resnet_frozen, train_resnet_ds,
    ↪ val_resnet_ds, "resnet50_frozen", epochs=25)

resnet_finetuned = keras.models.clone_model(resnet_frozen)
resnet_finetuned.set_weights(resnet_frozen.get_weights())
resnet_finetuned._name = "resnet50_finetuned"
resnet_finetuned = prepare_resnet_for_fine_tuning(resnet_finetuned)
history_resnet_finetuned = train_model(resnet_finetuned, train_resnet_ds,
    ↪ val_resnet_ds, "resnet50_finetuned", epochs=15)

```

Training baseline_cnn

Epoch 1/25

9/10 0s 40ms/step -
accuracy: 0.5657 - auc: 0.5767 - loss: 0.7088 - precision: 0.5849 - recall:
0.8627

Epoch 1: val_loss improved from None to 0.67567, saving model to C:\Users\adrian
\Documents\IBM_Projects\DeepLearning_Project\outputs\baseline_cnn.keras

Epoch 1: finished saving model to C:\Users\adrian\Documents\IBM_Projects\DeepLea
rning_Project\outputs\baseline_cnn.keras

10/10 1s 66ms/step -
accuracy: 0.5849 - auc: 0.5521 - loss: 0.6915 - precision: 0.6270 - recall:
0.8061 - val_accuracy: 0.6176 - val_auc: 0.7363 - val_loss: 0.6757 -
val_precision: 0.6176 - val_recall: 1.0000 - learning_rate: 0.0010

Epoch 2/25

9/10 0s 40ms/step -
accuracy: 0.5038 - auc: 0.4680 - loss: 0.7214 - precision: 0.5348 - recall:
0.8555

Epoch 2: val_loss improved from 0.67567 to 0.67497, saving model to C:\Users\adrian\Documents\IBM_Projects\DeepLearning_Project\outputs\baseline_cnn.keras

Epoch 2: finished saving model to C:\Users\adrian\Documents\IBM_Projects\DeepLearning_Project\outputs\baseline_cnn.keras

10/10 1s 52ms/step -

accuracy: 0.5849 - auc: 0.5274 - loss: 0.6924 - precision: 0.6111 - recall: 0.8980 - val_accuracy: 0.6176 - val_auc: 0.7161 - val_loss: 0.6750 - val_precision: 0.6176 - val_recall: 1.0000 - learning_rate: 0.0010

Epoch 3/25

10/10 0s 41ms/step -

accuracy: 0.6085 - auc: 0.5731 - loss: 0.6917 - precision: 0.6175 - recall: 0.9501

Epoch 3: val_loss did not improve from 0.67497

10/10 1s 51ms/step -

accuracy: 0.6352 - auc: 0.6050 - loss: 0.6873 - precision: 0.6370 - recall: 0.9490 - val_accuracy: 0.6176 - val_auc: 0.7582 - val_loss: 0.6755 - val_precision: 0.6176 - val_recall: 1.0000 - learning_rate: 0.0010

Epoch 4/25

10/10 0s 40ms/step -

accuracy: 0.6148 - auc: 0.6297 - loss: 0.7000 - precision: 0.6134 - recall: 0.9158

Epoch 4: val_loss did not improve from 0.67497

10/10 1s 50ms/step -

accuracy: 0.6352 - auc: 0.6321 - loss: 0.6831 - precision: 0.6493 - recall: 0.8878 - val_accuracy: 0.6765 - val_auc: 0.7289 - val_loss: 0.6757 - val_precision: 0.6786 - val_recall: 0.9048 - learning_rate: 0.0010

Epoch 5/25

10/10 0s 40ms/step -

accuracy: 0.6749 - auc: 0.6364 - loss: 0.6744 - precision: 0.6936 - recall: 0.8565

Epoch 5: val_loss improved from 0.67497 to 0.64981, saving model to C:\Users\adrian\Documents\IBM_Projects\DeepLearning_Project\outputs\baseline_cnn.keras

Epoch 5: finished saving model to C:\Users\adrian\Documents\IBM_Projects\DeepLearning_Project\outputs\baseline_cnn.keras

10/10 1s 52ms/step -

accuracy: 0.6730 - auc: 0.6344 - loss: 0.6789 - precision: 0.6742 - recall: 0.9082 - val_accuracy: 0.6765 - val_auc: 0.7363 - val_loss: 0.6498 - val_precision: 0.6562 - val_recall: 1.0000 - learning_rate: 0.0010

Epoch 6/25

9/10 0s 39ms/step -

accuracy: 0.6310 - auc: 0.6691 - loss: 0.6718 - precision: 0.6838 - recall: 0.7348

Epoch 6: val_loss did not improve from 0.64981

10/10 1s 49ms/step -

accuracy: 0.5912 - auc: 0.6100 - loss: 0.6738 - precision: 0.7037 - recall: 0.5816 - val_accuracy: 0.7059 - val_auc: 0.7271 - val_loss: 0.6541 -

```

val_precision: 0.7391 - val_recall: 0.8095 - learning_rate: 0.0010
Epoch 7/25
10/10          0s 40ms/step -
accuracy: 0.6215 - auc: 0.6494 - loss: 0.7063 - precision: 0.6484 - recall:
0.7041
Epoch 7: val_loss did not improve from 0.64981
10/10          1s 50ms/step -
accuracy: 0.5849 - auc: 0.6332 - loss: 0.6707 - precision: 0.7222 - recall:
0.5306 - val_accuracy: 0.5588 - val_auc: 0.7015 - val_loss: 0.6802 -
val_precision: 0.8000 - val_recall: 0.3810 - learning_rate: 0.0010
Epoch 8/25
 9/10          0s 40ms/step -
accuracy: 0.5731 - auc: 0.6629 - loss: 0.6611 - precision: 0.6728 - recall:
0.5221
Epoch 8: ReduceLROnPlateau reducing learning rate to 0.0003000000142492354.

Epoch 8: val_loss did not improve from 0.64981
10/10          1s 49ms/step -
accuracy: 0.6478 - auc: 0.7167 - loss: 0.6365 - precision: 0.7561 - recall:
0.6327 - val_accuracy: 0.6471 - val_auc: 0.7088 - val_loss: 0.6536 -
val_precision: 0.8000 - val_recall: 0.5714 - learning_rate: 0.0010
Epoch 9/25
10/10          0s 41ms/step -
accuracy: 0.6000 - auc: 0.6570 - loss: 0.6431 - precision: 0.7096 - recall:
0.6283
Epoch 9: val_loss improved from 0.64981 to 0.64934, saving model to C:\Users\adr
ian\Documents\IBM_Projects\DeepLearning_Project\outputs\baseline_cnn.keras

Epoch 9: finished saving model to C:\Users\adrian\Documents\IBM_Projects\DeepLea
rning_Project\outputs\baseline_cnn.keras
10/10          1s 53ms/step -
accuracy: 0.6541 - auc: 0.6940 - loss: 0.6380 - precision: 0.7363 - recall:
0.6837 - val_accuracy: 0.6471 - val_auc: 0.7143 - val_loss: 0.6493 -
val_precision: 0.8000 - val_recall: 0.5714 - learning_rate: 3.0000e-04
Epoch 10/25
 9/10          0s 40ms/step -
accuracy: 0.7151 - auc: 0.7217 - loss: 0.6332 - precision: 0.8054 - recall:
0.7346
Epoch 10: val_loss did not improve from 0.64934
10/10          1s 52ms/step -
accuracy: 0.7044 - auc: 0.7298 - loss: 0.6239 - precision: 0.7865 - recall:
0.7143 - val_accuracy: 0.6471 - val_auc: 0.7125 - val_loss: 0.6564 -
val_precision: 0.8000 - val_recall: 0.5714 - learning_rate: 3.0000e-04
Epoch 11/25
10/10          0s 42ms/step -
accuracy: 0.6111 - auc: 0.6501 - loss: 0.6811 - precision: 0.6774 - recall:
0.5556
Epoch 11: val_loss did not improve from 0.64934

```

10/10 1s 52ms/step -
accuracy: 0.5975 - auc: 0.6742 - loss: 0.6438 - precision: 0.7297 - recall:
0.5510 - val_accuracy: 0.5294 - val_auc: 0.7125 - val_loss: 0.6959 -
val_precision: 0.7778 - val_recall: 0.3333 - learning_rate: 3.0000e-04
Epoch 12/25
9/10 0s 41ms/step -
accuracy: 0.5747 - auc: 0.7147 - loss: 0.6289 - precision: 0.7666 - recall:
0.4337
Epoch 12: ReduceLROnPlateau reducing learning rate to 9.000000427477062e-05.

Epoch 12: val_loss did not improve from 0.64934
10/10 1s 51ms/step -
accuracy: 0.6289 - auc: 0.7341 - loss: 0.6102 - precision: 0.7910 - recall:
0.5408 - val_accuracy: 0.6176 - val_auc: 0.7234 - val_loss: 0.6613 -
val_precision: 0.7857 - val_recall: 0.5238 - learning_rate: 3.0000e-04
Epoch 13/25
9/10 0s 41ms/step -
accuracy: 0.6496 - auc: 0.7215 - loss: 0.5942 - precision: 0.8107 - recall:
0.6217
Epoch 13: val_loss improved from 0.64934 to 0.64789, saving model to C:\Users\adrian\Documents\IBM_Projects\DeepLearning_Project\outputs\baseline_cnn.keras

Epoch 13: finished saving model to C:\Users\adrian\Documents\IBM_Projects\DeepLearning_Project\outputs\baseline_cnn.keras
10/10 1s 53ms/step -
accuracy: 0.6478 - auc: 0.7364 - loss: 0.6062 - precision: 0.7763 - recall:
0.6020 - val_accuracy: 0.6471 - val_auc: 0.7253 - val_loss: 0.6479 -
val_precision: 0.8000 - val_recall: 0.5714 - learning_rate: 9.0000e-05
Epoch 14/25
10/10 0s 41ms/step -
accuracy: 0.6266 - auc: 0.7426 - loss: 0.6119 - precision: 0.6761 - recall:
0.6500
Epoch 14: val_loss did not improve from 0.64789
10/10 1s 52ms/step -
accuracy: 0.6478 - auc: 0.7196 - loss: 0.6072 - precision: 0.7333 - recall:
0.6735 - val_accuracy: 0.6471 - val_auc: 0.7253 - val_loss: 0.6502 -
val_precision: 0.8000 - val_recall: 0.5714 - learning_rate: 9.0000e-05
Epoch 15/25
9/10 0s 40ms/step -
accuracy: 0.7174 - auc: 0.7797 - loss: 0.5694 - precision: 0.8169 - recall:
0.7164
Epoch 15: val_loss did not improve from 0.64789
10/10 1s 49ms/step -
accuracy: 0.6981 - auc: 0.7736 - loss: 0.5812 - precision: 0.7976 - recall:
0.6837 - val_accuracy: 0.6471 - val_auc: 0.7289 - val_loss: 0.6549 -
val_precision: 0.8000 - val_recall: 0.5714 - learning_rate: 9.0000e-05
Epoch 16/25
10/10 0s 40ms/step -

accuracy: 0.6319 - auc: 0.6754 - loss: 0.6447 - precision: 0.7524 - recall: 0.6066

Epoch 16: ReduceLROnPlateau reducing learning rate to 2.700000040931627e-05.

Epoch 16: val_loss did not improve from 0.64789

10/10 1s 50ms/step -

accuracy: 0.6541 - auc: 0.7094 - loss: 0.6243 - precision: 0.7654 - recall: 0.6327 - val_accuracy: 0.6176 - val_auc: 0.7289 - val_loss: 0.6589 - val_precision: 0.7857 - val_recall: 0.5238 - learning_rate: 9.0000e-05

Epoch 17/25

9/10 0s 39ms/step -

accuracy: 0.7066 - auc: 0.7694 - loss: 0.5940 - precision: 0.7903 - recall: 0.6752

Epoch 17: val_loss did not improve from 0.64789

10/10 1s 49ms/step -

accuracy: 0.6792 - auc: 0.7495 - loss: 0.6000 - precision: 0.8052 - recall: 0.6327 - val_accuracy: 0.6176 - val_auc: 0.7271 - val_loss: 0.6602 - val_precision: 0.7857 - val_recall: 0.5238 - learning_rate: 2.7000e-05

Epoch 18/25

9/10 0s 40ms/step -

accuracy: 0.6449 - auc: 0.7042 - loss: 0.5994 - precision: 0.7803 - recall: 0.6139

Epoch 18: val_loss did not improve from 0.64789

10/10 1s 50ms/step -

accuracy: 0.6478 - auc: 0.7328 - loss: 0.5958 - precision: 0.7561 - recall: 0.6327 - val_accuracy: 0.6176 - val_auc: 0.7271 - val_loss: 0.6589 - val_precision: 0.7857 - val_recall: 0.5238 - learning_rate: 2.7000e-05

Epoch 18: early stopping

Restoring model weights from the end of the best epoch: 13.

Training resnet50_frozen

Epoch 1/25

10/10 0s 114ms/step -

accuracy: 0.5374 - auc: 0.5895 - loss: 0.9088 - precision: 0.5969 - recall: 0.5674

Epoch 1: val_loss improved from None to 0.50107, saving model to C:\Users\adrian\Documents\IBM_Projects\DeepLearning_Project\outputs\resnet50_frozen.keras

Epoch 1: finished saving model to C:\Users\adrian\Documents\IBM_Projects\DeepLearning_Project\outputs\resnet50_frozen.keras

10/10 5s 265ms/step -

accuracy: 0.6352 - auc: 0.6833 - loss: 0.7549 - precision: 0.7000 - recall: 0.7143 - val_accuracy: 0.7647 - val_auc: 0.8535 - val_loss: 0.5011 - val_precision: 0.8824 - val_recall: 0.7143 - learning_rate: 0.0010

Epoch 2/25

10/10 0s 116ms/step -

accuracy: 0.8217 - auc: 0.8903 - loss: 0.4579 - precision: 0.9352 - recall: 0.7473

Epoch 2: val_loss did not improve from 0.50107

10/10 2s 152ms/step -
accuracy: 0.8176 - auc: 0.8937 - loss: 0.4496 - precision: 0.9157 - recall:
0.7755 - val_accuracy: 0.7941 - val_auc: 0.8883 - val_loss: 0.5308 -
val_precision: 0.7500 - val_recall: 1.0000 - learning_rate: 0.0010
Epoch 3/25
10/10 0s 115ms/step -
accuracy: 0.8373 - auc: 0.8967 - loss: 0.6297 - precision: 0.8117 - recall:
0.9637
Epoch 3: val_loss did not improve from 0.50107
10/10 2s 152ms/step -
accuracy: 0.8428 - auc: 0.8894 - loss: 0.4908 - precision: 0.8687 - recall:
0.8776 - val_accuracy: 0.7353 - val_auc: 0.9158 - val_loss: 0.5964 -
val_precision: 0.9286 - val_recall: 0.6190 - learning_rate: 0.0010
Epoch 4/25
10/10 0s 117ms/step -
accuracy: 0.8261 - auc: 0.8922 - loss: 0.3907 - precision: 0.8936 - recall:
0.7917
Epoch 4: val_loss improved from 0.50107 to 0.34863, saving model to C:\Users\adrian\Documents\IBM_Projects\DeepLearning_Project\outputs\resnet50_frozen.keras

Epoch 4: finished saving model to C:\Users\adrian\Documents\IBM_Projects\DeepLearning_Project\outputs\resnet50_frozen.keras
10/10 2s 189ms/step -
accuracy: 0.8616 - auc: 0.9448 - loss: 0.2987 - precision: 0.9222 - recall:
0.8469 - val_accuracy: 0.8529 - val_auc: 0.9267 - val_loss: 0.3486 -
val_precision: 0.8636 - val_recall: 0.9048 - learning_rate: 0.0010
Epoch 5/25
10/10 0s 114ms/step -
accuracy: 0.8804 - auc: 0.9453 - loss: 0.3086 - precision: 0.9032 - recall:
0.9060
Epoch 5: val_loss did not improve from 0.34863
10/10 2s 150ms/step -
accuracy: 0.9057 - auc: 0.9533 - loss: 0.2847 - precision: 0.9278 - recall:
0.9184 - val_accuracy: 0.8529 - val_auc: 0.9176 - val_loss: 0.3605 -
val_precision: 0.8636 - val_recall: 0.9048 - learning_rate: 0.0010
Epoch 6/25
10/10 0s 116ms/step -
accuracy: 0.9177 - auc: 0.9728 - loss: 0.2258 - precision: 0.9606 - recall:
0.9008
Epoch 6: val_loss did not improve from 0.34863
10/10 2s 152ms/step -
accuracy: 0.8931 - auc: 0.9547 - loss: 0.2718 - precision: 0.9355 - recall:
0.8878 - val_accuracy: 0.8235 - val_auc: 0.9212 - val_loss: 0.3632 -
val_precision: 0.8947 - val_recall: 0.8095 - learning_rate: 0.0010
Epoch 7/25
10/10 0s 115ms/step -
accuracy: 0.8789 - auc: 0.9479 - loss: 0.2861 - precision: 0.9068 - recall:
0.8714

Epoch 7: ReduceLROnPlateau reducing learning rate to 0.0003000000142492354.

Epoch 7: val_loss did not improve from 0.34863

10/10 2s 152ms/step -

accuracy: 0.8616 - auc: 0.9436 - loss: 0.2989 - precision: 0.9222 - recall:
0.8469 - val_accuracy: 0.8824 - val_auc: 0.9267 - val_loss: 0.3490 -
val_precision: 0.8696 - val_recall: 0.9524 - learning_rate: 0.0010

Epoch 8/25

10/10 0s 115ms/step -

accuracy: 0.9118 - auc: 0.9673 - loss: 0.2535 - precision: 0.9249 - recall:
0.9260

Epoch 8: val_loss did not improve from 0.34863

10/10 2s 153ms/step -

accuracy: 0.9182 - auc: 0.9747 - loss: 0.2220 - precision: 0.9293 - recall:
0.9388 - val_accuracy: 0.8824 - val_auc: 0.9304 - val_loss: 0.3561 -
val_precision: 0.8696 - val_recall: 0.9524 - learning_rate: 3.0000e-04

Epoch 9/25

10/10 0s 116ms/step -

accuracy: 0.8876 - auc: 0.9624 - loss: 0.2436 - precision: 0.9144 - recall:
0.9084

Epoch 9: val_loss did not improve from 0.34863

10/10 2s 152ms/step -

accuracy: 0.9057 - auc: 0.9702 - loss: 0.2271 - precision: 0.9192 - recall:
0.9286 - val_accuracy: 0.7941 - val_auc: 0.9267 - val_loss: 0.3595 -
val_precision: 0.8500 - val_recall: 0.8095 - learning_rate: 3.0000e-04

Epoch 9: early stopping

Restoring model weights from the end of the best epoch: 4.

Training resnet50_finetuned

Epoch 1/15

10/10 0s 155ms/step -

accuracy: 0.9267 - auc: 0.9658 - loss: 0.2421 - precision: 0.9481 - recall:
0.9408

Epoch 1: val_loss improved from None to 0.37513, saving model to C:\Users\adrian
\Documents\IBM_Projects\DeepLearning_Project\outputs\resnet50_finetuned.keras

Epoch 1: finished saving model to C:\Users\adrian\Documents\IBM_Projects\DeepLea
rning_Project\outputs\resnet50_finetuned.keras

10/10 6s 326ms/step -

accuracy: 0.8616 - auc: 0.9420 - loss: 0.3331 - precision: 0.8654 - recall:
0.9184 - val_accuracy: 0.8529 - val_auc: 0.9286 - val_loss: 0.3751 -
val_precision: 0.9000 - val_recall: 0.8571 - learning_rate: 1.0000e-05

Epoch 2/15

10/10 0s 158ms/step -

accuracy: 0.8825 - auc: 0.9393 - loss: 0.4097 - precision: 0.9595 - recall:
0.8258

Epoch 2: val_loss did not improve from 0.37513

10/10 2s 196ms/step -

accuracy: 0.8491 - auc: 0.9537 - loss: 0.3531 - precision: 0.9868 - recall:

0.7653 - val_accuracy: 0.8529 - val_auc: 0.9286 - val_loss: 0.3909 -
val_precision: 0.9444 - val_recall: 0.8095 - learning_rate: 1.0000e-05
Epoch 3/15
10/10 0s 161ms/step -
accuracy: 0.8577 - auc: 0.9453 - loss: 0.2981 - precision: 0.8710 - recall:
0.8996
Epoch 3: val_loss improved from 0.37513 to 0.32856, saving model to C:\Users\adrian\Documents\IBM_Projects\DeepLearning_Project\outputs\resnet50_finetuned.keras

Epoch 3: finished saving model to C:\Users\adrian\Documents\IBM_Projects\DeepLearning_Project\outputs\resnet50_finetuned.keras
10/10 3s 259ms/step -
accuracy: 0.8679 - auc: 0.9281 - loss: 0.3703 - precision: 0.8667 - recall:
0.9286 - val_accuracy: 0.8529 - val_auc: 0.9377 - val_loss: 0.3286 -
val_precision: 0.8636 - val_recall: 0.9048 - learning_rate: 1.0000e-05
Epoch 4/15
10/10 0s 160ms/step -
accuracy: 0.9557 - auc: 0.9859 - loss: 0.1719 - precision: 0.9714 - recall:
0.9626
Epoch 4: val_loss did not improve from 0.32856
10/10 2s 196ms/step -
accuracy: 0.9434 - auc: 0.9761 - loss: 0.2136 - precision: 0.9588 - recall:
0.9490 - val_accuracy: 0.8824 - val_auc: 0.9304 - val_loss: 0.3694 -
val_precision: 0.9474 - val_recall: 0.8571 - learning_rate: 1.0000e-05
Epoch 5/15
10/10 0s 158ms/step -
accuracy: 0.9392 - auc: 0.9842 - loss: 0.1668 - precision: 0.9569 - recall:
0.9392
Epoch 5: val_loss did not improve from 0.32856
10/10 2s 196ms/step -
accuracy: 0.9308 - auc: 0.9793 - loss: 0.1825 - precision: 0.9780 - recall:
0.9082 - val_accuracy: 0.8824 - val_auc: 0.9359 - val_loss: 0.3488 -
val_precision: 0.9474 - val_recall: 0.8571 - learning_rate: 1.0000e-05
Epoch 6/15
10/10 0s 160ms/step -
accuracy: 0.9384 - auc: 0.9807 - loss: 0.1983 - precision: 0.9360 - recall:
0.9678
Epoch 6: val_loss improved from 0.32856 to 0.29384, saving model to C:\Users\adrian\Documents\IBM_Projects\DeepLearning_Project\outputs\resnet50_finetuned.keras

Epoch 6: finished saving model to C:\Users\adrian\Documents\IBM_Projects\DeepLearning_Project\outputs\resnet50_finetuned.keras
10/10 3s 258ms/step -
accuracy: 0.9371 - auc: 0.9842 - loss: 0.1835 - precision: 0.9314 - recall:
0.9694 - val_accuracy: 0.9118 - val_auc: 0.9505 - val_loss: 0.2938 -
val_precision: 0.9091 - val_recall: 0.9524 - learning_rate: 1.0000e-05
Epoch 7/15
10/10 0s 160ms/step -

accuracy: 0.9224 - auc: 0.9754 - loss: 0.1950 - precision: 0.9317 - recall: 0.9449

Epoch 7: val_loss improved from 0.29384 to 0.28399, saving model to C:\Users\adrian\Documents\IBM_Projects\DeepLearning_Project\outputs\resnet50_finetuned.keras

Epoch 7: finished saving model to C:\Users\adrian\Documents\IBM_Projects\DeepLearning_Project\outputs\resnet50_finetuned.keras

10/10 3s 256ms/step -

accuracy: 0.9434 - auc: 0.9829 - loss: 0.1617 - precision: 0.9495 - recall: 0.9592 - val_accuracy: 0.8824 - val_auc: 0.9524 - val_loss: 0.2840 - val_precision: 0.9048 - val_recall: 0.9048 - learning_rate: 1.0000e-05

Epoch 8/15

10/10 0s 159ms/step -

accuracy: 0.9563 - auc: 0.9949 - loss: 0.1027 - precision: 0.9686 - recall: 0.9567

Epoch 8: val_loss did not improve from 0.28399

10/10 2s 197ms/step -

accuracy: 0.9371 - auc: 0.9917 - loss: 0.1239 - precision: 0.9583 - recall: 0.9388 - val_accuracy: 0.8824 - val_auc: 0.9579 - val_loss: 0.2882 - val_precision: 0.9474 - val_recall: 0.8571 - learning_rate: 1.0000e-05

Epoch 9/15

10/10 0s 158ms/step -

accuracy: 0.9687 - auc: 0.9973 - loss: 0.0801 - precision: 0.9798 - recall: 0.9686

Epoch 9: val_loss improved from 0.28399 to 0.27697, saving model to C:\Users\adrian\Documents\IBM_Projects\DeepLearning_Project\outputs\resnet50_finetuned.keras

Epoch 9: finished saving model to C:\Users\adrian\Documents\IBM_Projects\DeepLearning_Project\outputs\resnet50_finetuned.keras

10/10 3s 253ms/step -

accuracy: 0.9245 - auc: 0.9891 - loss: 0.1368 - precision: 0.9778 - recall: 0.8980 - val_accuracy: 0.9118 - val_auc: 0.9634 - val_loss: 0.2770 - val_precision: 0.9091 - val_recall: 0.9524 - learning_rate: 1.0000e-05

Epoch 10/15

10/10 0s 160ms/step -

accuracy: 0.9626 - auc: 0.9924 - loss: 0.1262 - precision: 0.9785 - recall: 0.9647

Epoch 10: val_loss did not improve from 0.27697

10/10 2s 197ms/step -

accuracy: 0.9560 - auc: 0.9875 - loss: 0.1535 - precision: 0.9505 - recall: 0.9796 - val_accuracy: 0.9118 - val_auc: 0.9634 - val_loss: 0.2905 - val_precision: 0.9091 - val_recall: 0.9524 - learning_rate: 1.0000e-05

Epoch 11/15

10/10 0s 161ms/step -

accuracy: 0.9618 - auc: 0.9944 - loss: 0.0875 - precision: 0.9914 - recall: 0.9460

Epoch 11: val_loss did not improve from 0.27697

10/10 2s 200ms/step -

```

accuracy: 0.9497 - auc: 0.9895 - loss: 0.1269 - precision: 0.9787 - recall:
0.9388 - val_accuracy: 0.8824 - val_auc: 0.9451 - val_loss: 0.3304 -
val_precision: 0.9474 - val_recall: 0.8571 - learning_rate: 1.0000e-05
Epoch 12/15
10/10          0s 163ms/step -
accuracy: 0.9555 - auc: 0.9960 - loss: 0.0936 - precision: 0.9780 - recall:
0.9453
Epoch 12: ReduceLROnPlateau reducing learning rate to 2.9999999242136253e-06.

Epoch 12: val_loss did not improve from 0.27697
10/10          2s 202ms/step -
accuracy: 0.9560 - auc: 0.9946 - loss: 0.0981 - precision: 0.9892 - recall:
0.9388 - val_accuracy: 0.8824 - val_auc: 0.9670 - val_loss: 0.3034 -
val_precision: 0.8696 - val_recall: 0.9524 - learning_rate: 1.0000e-05
Epoch 13/15
10/10          0s 157ms/step -
accuracy: 0.9667 - auc: 0.9923 - loss: 0.1053 - precision: 0.9712 - recall:
0.9744
Epoch 13: val_loss did not improve from 0.27697
10/10          2s 195ms/step -
accuracy: 0.9434 - auc: 0.9883 - loss: 0.1427 - precision: 0.9495 - recall:
0.9592 - val_accuracy: 0.8824 - val_auc: 0.9652 - val_loss: 0.3062 -
val_precision: 0.8696 - val_recall: 0.9524 - learning_rate: 3.0000e-06
Epoch 14/15
10/10          0s 157ms/step -
accuracy: 0.9879 - auc: 0.9970 - loss: 0.0887 - precision: 1.0000 - recall:
0.9817
Epoch 14: val_loss did not improve from 0.27697
10/10          2s 195ms/step -
accuracy: 0.9874 - auc: 0.9980 - loss: 0.0758 - precision: 1.0000 - recall:
0.9796 - val_accuracy: 0.9118 - val_auc: 0.9652 - val_loss: 0.2975 -
val_precision: 0.9091 - val_recall: 0.9524 - learning_rate: 3.0000e-06
Epoch 14: early stopping
Restoring model weights from the end of the best epoch: 9.

```

1.10 Evaluation and Model Selection

The validation set is used for model comparison and then we use the test set once for the selected final model. Below we have an analysis on the performance of the models with an explanation for the selected model.

```

[18]: def predict_probabilities(model: keras.Model, dataset) -> np.ndarray:
        return model.predict(dataset, verbose=0).reshape(-1)

def evaluate_predictions(y_true, y_prob, threshold: float = 0.50) -> dict:
    y_pred = (y_prob >= threshold).astype(int)
    metrics = {

```

```

        "accuracy": accuracy_score(y_true, y_pred),
        "precision": precision_score(y_true, y_pred, zero_division=0),
        "recall": recall_score(y_true, y_pred, zero_division=0),
        "f1": f1_score(y_true, y_pred, zero_division=0),
        "roc_auc": roc_auc_score(y_true, y_prob) if len(np.unique(y_true)) == 2
    else np.nan,
    }
    return metrics

def evaluate_model(model_name: str, model: keras.Model, dataset, df: pd.
    DataFrame) -> dict:
    y_true = df["label"].to_numpy().astype(int)
    y_prob = predict_probabilities(model, dataset)
    metrics = evaluate_predictions(y_true, y_prob)
    metrics["model"] = model_name
    return metrics

validation_results = pd.DataFrame(
    [
        evaluate_model("Baseline CNN", baseline_cnn, val_cnn_ds, val_df),
        evaluate_model("Frozen ResNet50", resnet_frozen, val_resnet_ds, val_df),
        evaluate_model("Fine-tuned ResNet50", resnet_finetuned, val_resnet_ds,
    val_df),
    ]
).sort_values(["f1", "recall", "roc_auc"], ascending=False)

validation_results.to_csv(OUTPUT_DIR / "model_validation_comparison.csv",
    index=False)
display(validation_results)

best_model_name = validation_results.iloc[0]["model"]
model_lookup = {
    "Baseline CNN": (baseline_cnn, test_cnn_ds),
    "Frozen ResNet50": (resnet_frozen, test_resnet_ds),
    "Fine-tuned ResNet50": (resnet_finetuned, test_resnet_ds),
}
best_model, best_test_ds = model_lookup[best_model_name]
print(f"Recommended model based on validation F1/recall: {best_model_name}")

```

WARNING:tensorflow:5 out of the last 7 calls to <function TensorFlowTrainer.make_predict_function.<locals>.one_step_on_data_distributed at 0x000002202B9A8720> triggered tf.function retracing. Tracing is expensive and the excessive number of tracings could be due to (1) creating @tf.function repeatedly in a loop, (2) passing tensors with different shapes, (3) passing Python objects instead of tensors. For (1), please define your @tf.function

outside of the loop. For (2), @tf.function has reduce_retracing=True option that can avoid unnecessary retracing. For (3), please refer to https://www.tensorflow.org/guide/function#controlling_retracing and https://www.tensorflow.org/api_docs/python/tf/function for more details.

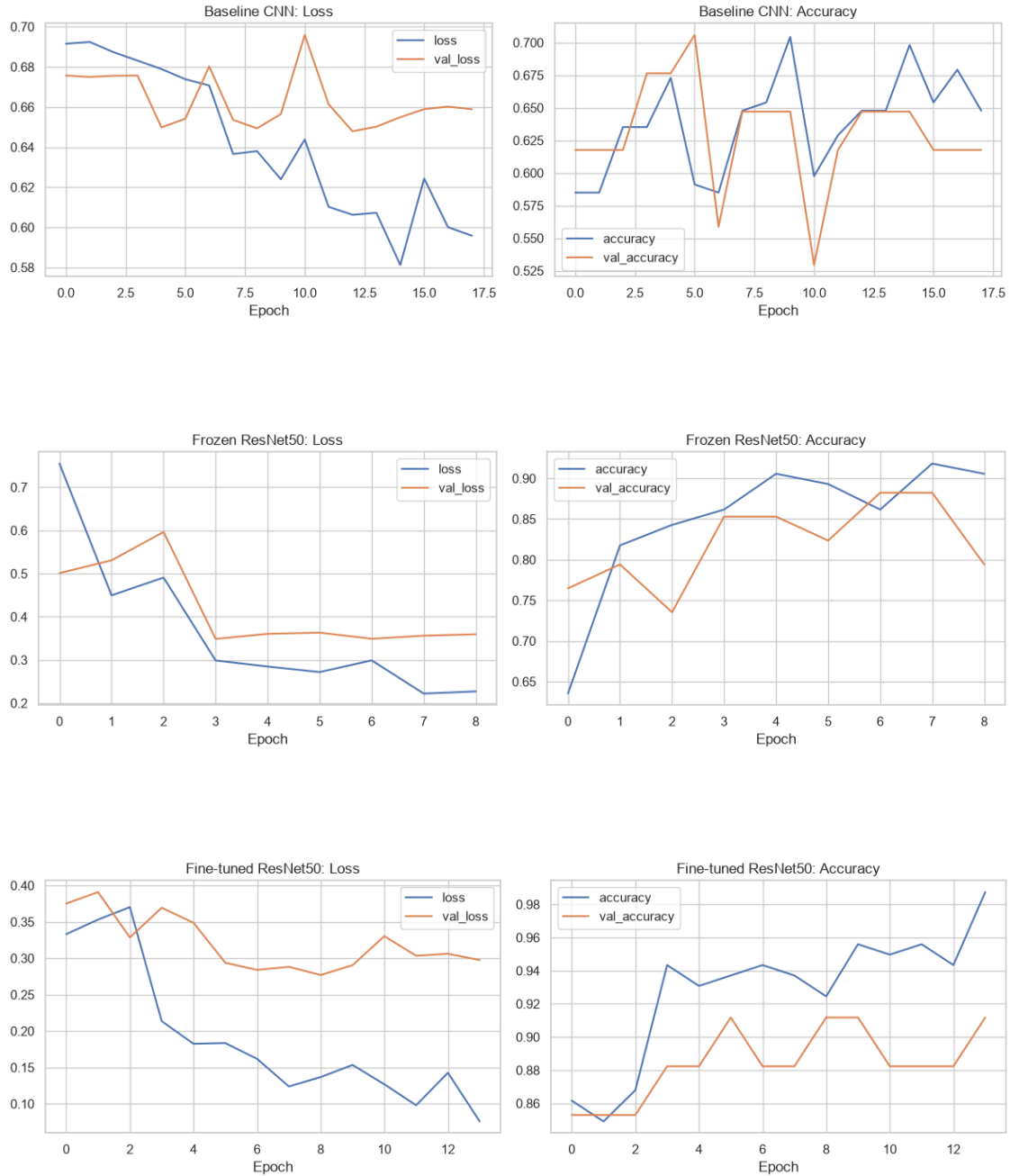
WARNING:tensorflow:6 out of the last 9 calls to <function TensorFlowTrainer.make_predict_function.<locals>.one_step_on_data_distributed at 0x000002202B9A8720> triggered tf.function retracing. Tracing is expensive and the excessive number of tracings could be due to (1) creating @tf.function repeatedly in a loop, (2) passing tensors with different shapes, (3) passing Python objects instead of tensors. For (1), please define your @tf.function outside of the loop. For (2), @tf.function has reduce_retracing=True option that can avoid unnecessary retracing. For (3), please refer to https://www.tensorflow.org/guide/function#controlling_retracing and https://www.tensorflow.org/api_docs/python/tf/function for more details.

	accuracy	precision	recall	f1	roc_auc	model
2	0.911765	0.909091	0.952381	0.930233	0.963370	Fine-tuned ResNet50
1	0.852941	0.863636	0.904762	0.883721	0.926740	Frozen ResNet50
0	0.647059	0.800000	0.571429	0.666667	0.721612	Baseline CNN

Recommended model based on validation F1/recall: Fine-tuned ResNet50

```
[19]: def plot_history(history, title: str):
    history_df = pd.DataFrame(history.history)
    fig, axes = plt.subplots(1, 2, figsize=(12, 4))
    history_df[["loss", "val_loss"]].plot(ax=axes[0])
    axes[0].set_title(f"{title}: Loss")
    axes[0].set_xlabel("Epoch")
    history_df[["accuracy", "val_accuracy"]].plot(ax=axes[1])
    axes[1].set_title(f"{title}: Accuracy")
    axes[1].set_xlabel("Epoch")
    plt.tight_layout()
    plt.savefig(OUTPUT_DIR / f"{title.lower().replace(' ', '_')}_training_curves.png", dpi=160, bbox_inches="tight")
    plt.show()

plot_history(history_baseline, "Baseline CNN")
plot_history(history_resnet_frozen, "Frozen ResNet50")
plot_history(history_resnet_finetuned, "Fine-tuned ResNet50")
```



We can see clearly that our baseline CNN model performs quite poorly. While its loss was decreasing, the validation loss kept stagnant throughout the training epochs. The model was not learning any particular features and was thus unable to fit to the validation set even as its loss started to decrease on the training set, a classic indication of overfitting. The accuracy is heavily unstable. Sometimes it performs better than guessing, otherwise it performs as well as guessing a yes or no blindly, so the model's performance is overall poor.

The ResNet50 model performs much better, but after 3 epochs, the validation loss comes to a halt

at around 0.35, with the training loss decreasing slowly. The model was starting to overfit, and the early stopping method terminated the training process before going through all of the epochs, stopping at 8 epochs compared to the 17-18 epochs that the CNN trained for. Its accuracy is considerably better than the CNN, steadily increasing at around 80-90% on the validation set and stabilizing in that general range.

Lastly, our finetuned ResNet50 performs much better than both of the previous models. It starts at both a much lower loss and higher accuracy than the rest. The loss decreases very steadily and the validation loss plateaus after 6-7 epochs. The early stopping mechanism halts the training at 12 epochs, effectively saving us a lot of time and avoiding overfitting the model. The accuracy of the model also steadily increases, with validation accuracy also plateauing at around 6-7 epochs, sitting at around 90% in general

Therefore, based on the performance, as well as the training time, we select our Finetuned ResNet50 model as the best and final candidate for our problem.

```
[20]: test_y_true = test_df["label"].to_numpy().astype(int)
test_y_prob = predict_probabilities(best_model, best_test_ds)
test_y_pred = (test_y_prob >= 0.50).astype(int)

test_metrics = pd.DataFrame([evaluate_predictions(test_y_true, test_y_prob)])
test_metrics.insert(0, "model", best_model_name)
test_metrics.to_csv(OUTPUT_DIR / "selected_model_test_metrics.csv", index=False)
display(test_metrics)

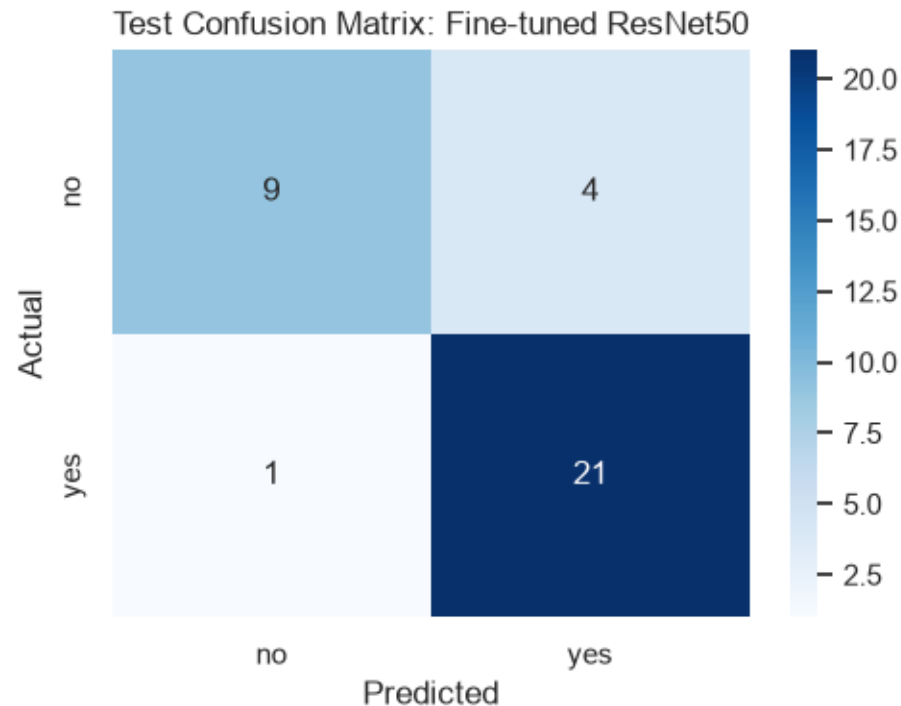
print(classification_report(test_y_true, test_y_pred, target_names=CLASS_NAMES,
    ↪zero_division=0))

cm = confusion_matrix(test_y_true, test_y_pred)
plt.figure(figsize=(5, 4))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", xticklabels=CLASS_NAMES,
    ↪yticklabels=CLASS_NAMES)
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title(f"Test Confusion Matrix: {best_model_name}")
plt.tight_layout()
plt.savefig(OUTPUT_DIR / "selected_model_test_confusion_matrix.png", dpi=160,
    ↪bbox_inches="tight")
plt.show()

best_model.save(OUTPUT_DIR / "selected_brain_tumor_model.keras")
```

	model	accuracy	precision	recall	f1	roc_auc
0	Fine-tuned ResNet50	0.857143	0.84	0.954545	0.893617	0.891608
		precision	recall	f1-score	support	
	no	0.90	0.69	0.78	13	
	yes	0.84	0.95	0.89	22	

accuracy			0.86	35
macro avg	0.87	0.82	0.84	35
weighted avg	0.86	0.86	0.85	35



We can see how well the model performs. For a small dataset, this performance is quite acceptable, but it does struggle with creating false positives, as evidenced by the lower recall of 0.69 and the proportion of false positives compared to false negatives. The model is good at detecting real positives, and struggles slightly with negative examples. This is due partly to the fact that we had less images without tumors than with tumors. The model had naturally less examples to learn from.

1.11 Explainability with Grad-CAM

Grad-CAM is used for the selected ResNet50 model and it works with ResNet based models. Note that these visualizations are approximate explanations and should not be treated as clinical evidence.

The idea behind Grad-CAM is to show a heatmap of where the model looked for when trying to find the specific features that make up a tumor. It takes the last convolutional layer and looks at what features that layer looked at. This is done for both comfort and computational efficiency, but also because the last layer is the one that should have used all of the previous layers to move from generic and broad features like a shape or an edge to the actual components that make up a tumor. It is a great tool to visualize where our Finetuned ResNet50 model is looking when trying to find

the tumor. One thing to keep in mind that this doesn't show whether the model found a tumor or not and whether it is correct. It simply displays the places that the model considered as most important to look at when determining if an image showed a tumor or not.

```
[21]: def find_last_conv_layer(model: keras.Model) -> str:
    for layer in reversed(model.layers):
        if isinstance(layer, keras.Model):
            for sublayer in reversed(layer.layers):
                if isinstance(sublayer, layers.Conv2D):
                    return sublayer.name
        if isinstance(layer, layers.Conv2D):
            return layer.name
    raise ValueError("No Conv2D layer found for Grad-CAM.")

def preprocess_single_image(path: str, model_family: str) -> tf.Tensor:
    image = tf.io.read_file(path)
    image = tf.io.decode_image(image, channels=3, expand_animations=False)
    image = tf.image.resize(image, IMG_SIZE)
    image = tf.cast(image, tf.float32)
    if model_family == "resnet":
        image = preprocess_input(image)
    else:
        image = image / 255.0
    return tf.expand_dims(image, axis=0)

def make_gradcam_heatmap(model: keras.Model, image_tensor: tf.Tensor,
    ↪last_conv_layer_name: str) -> np.ndarray:
    base_model = model.get_layer("resnet50")
    conv_layer = base_model.get_layer(last_conv_layer_name)
    base_index = model.layers.index(base_model)
    head_layers = model.layers[base_index + 1:]
    activation_model = keras.Model(
        inputs=base_model.input,
        outputs=[conv_layer.output, base_model.output],
    )

    with tf.GradientTape() as tape:
        conv_outputs, x = activation_model(image_tensor)
        for layer in head_layers:
            try:
                x = layer(x, training=False)
            except TypeError:
                x = layer(x)
        loss = x[:, 0]
```

```

gradients = tape.gradient(loss, conv_outputs)
pooled_gradients = tf.reduce_mean(gradients, axis=(0, 1, 2))
conv_outputs = conv_outputs[0]
heatmap = conv_outputs @ pooled_gradients[..., tf.newaxis]
heatmap = tf.squeeze(heatmap)
max_value = tf.math.reduce_max(heatmap)
heatmap = tf.where(max_value > 0, tf.maximum(heatmap, 0) / max_value, tf.
↳zeros_like(heatmap))
    return heatmap.numpy()

def plot_gradcam_examples(model: keras.Model, df: pd.DataFrame, model_name:
↳str, max_examples: int = 6):
    if "ResNet50" not in model_name:
        print("Grad-CAM block is configured for ResNet50 models. Skipping for
↳baseline CNN selection.")
        return

    last_conv_name = find_last_conv_layer(model.get_layer("resnet50"))
    example_df = df.copy().head(max_examples).reset_index(drop=True)
    fig, axes = plt.subplots(len(example_df), 2, figsize=(8, 3 *
↳len(example_df)))
    if len(example_df) == 1:
        axes = np.expand_dims(axes, axis=0)

    for idx, row in example_df.iterrows():
        image_tensor = preprocess_single_image(row["path"], "resnet")
        heatmap = make_gradcam_heatmap(model, image_tensor, last_conv_name)
        with Image.open(row["path"]) as raw_image:
            raw_image = ImageOps.exif_transpose(raw_image).convert("RGB").
↳resize(IMG_SIZE)

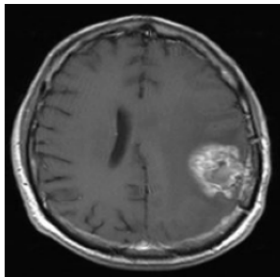
        axes[idx, 0].imshow(raw_image)
        axes[idx, 0].set_title(f"Actual: {row['label_name']}")
        axes[idx, 0].axis("off")
        axes[idx, 1].imshow(raw_image)
        axes[idx, 1].imshow(heatmap, cmap="jet", alpha=0.40, extent=(0,
↳IMG_SIZE[0], IMG_SIZE[1], 0))
        axes[idx, 1].set_title("Grad-CAM Overlay")
        axes[idx, 1].axis("off")

    plt.tight_layout()
    plt.savefig(OUTPUT_DIR / "gradcam_examples.png", dpi=160,
↳bbox_inches="tight")
    plt.show()

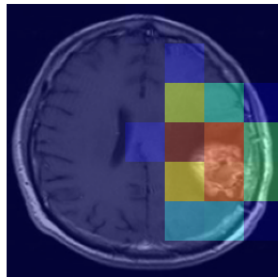
```

```
plot_gradcam_examples(best_model, test_df, best_model_name)
```

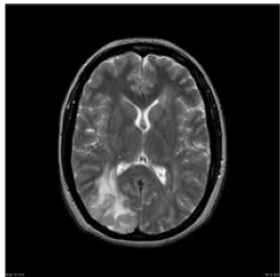
Actual: yes



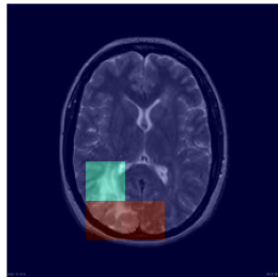
Grad-CAM Overlay



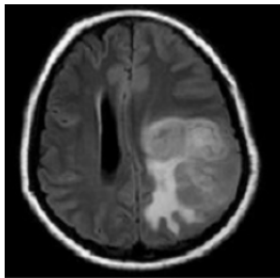
Actual: no



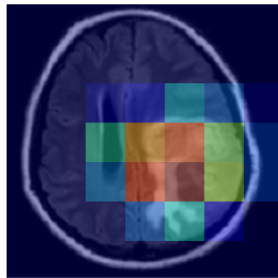
Grad-CAM Overlay



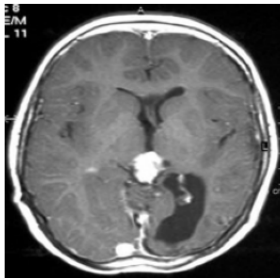
Actual: yes



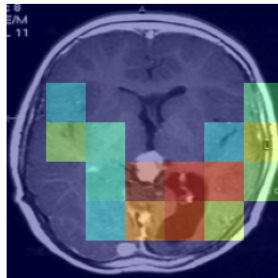
Grad-CAM Overlay



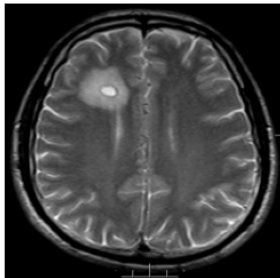
Actual: yes



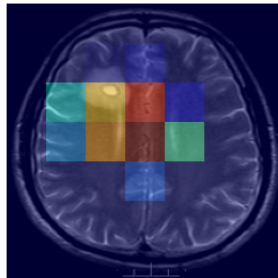
Grad-CAM Overlay



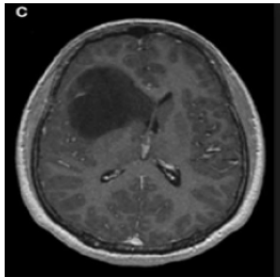
Actual: yes



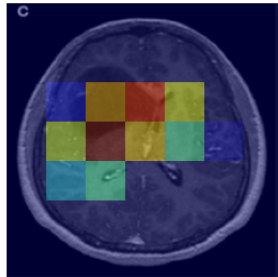
Grad-CAM Overlay



Actual: yes



Grad-CAM Overlay



We can see that the model is, in general, doing a great job at finding those features that make up a tumor.

1.12 Key Findings and Limitations

- Our Finetuned ResNet50 model shows great diagnostic performance in the preliminary tests we performed, having learned with a considerably good degree of accuracy the features that make up a tumor and being able to spot them in grayscale brain MRI scans. It shows a high F1-score of 0.81 and a high recall score of 0.95, with general accuracy sitting at around 0.85.
- However, due to limitations on dataset size, the model shows slight problems with identifying negatives examples, thus sometimes creating false positives.

The main limitations that we need to be aware of are the following:

- The dataset is small after duplicate removal, with 228 unique images.
- The classes are imbalanced, with more tumor images than no-tumor images.
- There is no patient metadata, scanner metadata, tumor type, tumor grade, or external validation cohort.
- ResNet50 is pretrained on natural images, so there is a domain gap between ImageNet and MRI scans.
- In the end, this model should not be presented as a clinical diagnostic tool, but it is a good baseline.

1.13 Next Steps

For future work and analysis, we recommend the following steps:

- Validate the selected model on an independent external MRI dataset from a different source.
- Expand the dataset with more no-tumor examples and more diverse scanner/acquisition conditions.
- Add clinical metadata if available, including patient age, tumor type, tumor grade, and scanner protocol.
- Compare ResNet50 with one additional modern medical-imaging-friendly architecture.
- Review Grad-CAM outputs with a domain expert to assess whether the model is focusing on clinically plausible regions.
- Consider tumor localization or segmentation as a future extension if annotated masks become available.